MedAssist AI

Implementation, local verification, UI review, and release limits

Developed by J Ganesh kumar reddy. Continuation and verification dated 30 October 2026. This educational project uses a responsive workspace and explicit evidence labels.

Executive summary

The application includes rotating and revocable sessions, server-owned medical routing, PubMed retrieval, explicit evidence fallbacks, retry-safe saves, encrypted personal provider keys, and opt-in bounded past-chat context. This continuation corrected misleading privacy copy, removed generic follow-ups after safety redirects, and changed calculators and medication-pair flags to educational estimates and cautions. Local tests exercise these paths but do not establish clinical validity.

The 30 October headless browser pass ran against frontend/dist on port 4173 with synthetic providers and a local email inbox. HTTP, authentication, MongoDB and WebSockets were real in the isolated test stack. The browser verified registration and email confirmation using a disposable account. Automated disposable-account tests cover password changes and permanent deletion; browser checks cover their forms and guards. These checks do not establish clinical correctness, model accuracy or completion of public deployment.

Verification area	Recorded result
Backend service/API/database/WebSocket tests	64 passed
Frontend component/client tests	43 passed
Dataset / LoRA tests	13 passed / 6 passed
Headless browser workflow checklist	20 of 20 passed
Built-output HTTP checks	7 passed
Frontend lint and production build	Passed
backend production npm audit	0 reported vulnerabilities
frontend production npm audit	0 reported vulnerabilities
Docker, real Gemini and email	Not exercised locally; host/credentials required

No external messages were sent. Local test messages exercised the chat with synthetic services. No LoRA model weights were downloaded, no GPU training ran and no adapter was published. The source CSV remains unchanged. Docker, public TLS, real mailbox delivery, credentialed generation and backup restore are still release checks. The included nginx container does not terminate TLS.

1. Architecture, API and data contracts

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Runtime Overview & Engineering Invariants

MedAssist AI runs as a web application with a **React 19 Single Page Application (SPA)**, an **Express / Node.js 24** backend, an authenticated **MongoDB 7** database, a **WebSocket gateway (/ws)**, and optional external integrations. The local verification environment uses synthetic AI, PubMed, and email responses; production hardening and clinical validation remain open.

2. Runtime Capability Mindmap

3. Database Models & Schema Specifications

The database layer runs on MongoDB 7 via Mongoose. All models enforce strict schema boundaries, validation rules, and indexing.

3.1 User Model (backend/models/userModel.js)

Manages user authentication, single-device session versioning, and provider preferences:

3.2 Provider Credential Model (backend/models/providerCredentialModel.js)

Stores user-provided API keys encrypted via **AES-256-GCM**. Raw keys are never stored in plaintext:

3.3 Ephemeral Generation Model (backend/models/generationModel.js)

Buffers in-flight and completed generation payloads. Enforces a **24-hour Time-to-Live (TTL)** expiration:

3.4 Chat & Message Model (backend/models/chatModel.js)

Stores permanently committed conversation history, PubMed bibliographic citations, and verification outcomes:

4. Multi-Provider AI Architecture & Personal Key Vault

MedAssist AI supports multi-provider generation with operator-defined fallbacks or authenticated user-supplied personal keys:

4.1 Cryptographic Key Vault Implementation (backend/services/providerCredentials.js)

5. Google OAuth 2.0 with PKCE Flow (backend/controllers/oauthController.js)

MedAssist AI implements standard OpenID Connect with **Proof Key for Code Exchange (PKCE)** using SHA-256 code challenges (S256):

6. Comprehensive REST API Specifications with JSON Payloads

6.1 Authentication Routes (/api/auth/*)

POST /api/auth/register

- **Request Body**:

json {

```
  "email": "student@example.com",
  "password": "SecurePassword123!"
```

}

- **Response 201 Created**:

json {

```
  "success": true,
  "message": "Account registered. Please check your email to verify your account."
}
```

POST /api/auth/login

- **Request Body**:

json {

```
  "email": "student@example.com",
  "password": "SecurePassword123!"
}
```

- **Response 200 OK** (Sets accessToken and refreshToken HttpOnly cookies):

json {

```
  "user": {
  "id": "67215f9b4c68832a8b9f1234",
  "email": "student@example.com",
  "isVerified": true
  }
}
```

GET /api/auth/check

- **Response 200 OK**:

json {

```
  "user": {
  "id": "67215f9b4c68832a8b9f1234",
  "email": "student@example.com"
  }
}
```

POST /api/auth/refresh-token

- **Request**: Cookie refreshToken

- **Response 200 OK**:

json {

```
  "status": "refreshed"
}
```

6.2 User Settings & Personal Key Vault (/api/settings/*)

GET /api/settings

- **Response 200 OK**:

json {

```
  "providerChoice": "deployment",
  "answerStyle": "balanced",
  "audienceProfile": "general",
  "usePastChats": false,
  "keyStorageAvailable": true,
  "personalKeys": [
  {
  "provider": "gemini",
  "lastFour": "1234",
  "updatedAt": "2026-10-29T10:15:30.000Z"
  }
  ],
  "providerChoices": [
  { "id": "deployment", "provider": "gemini", "source": "deployment", "available": true },
  { "id": "deployment-groq", "provider": "groq", "source": "deployment", "available": true },
  { "id": "personal-gemini", "provider": "gemini", "source": "personal", "available": true },
  { "id": "personal-groq", "provider": "groq", "source": "personal", "available": false }
  ]
}
```


PUT /api/settings/keys/:provider

- **Request Body**:

json {


```
  "apiKey": "AIzaSyD-PersonalSecretKey1234"
```


}

- **Response 200 OK**: Updated settings payload matching GET /api/settings.

PATCH /api/settings

- Accepts any subset of providerChoice, answerStyle, audienceProfile, and usePastChats. A newly selected provider must have an available deployment credential or a saved personal key. Cross-chat context is off by default and uses bounded word-overlap retrieval from the signed-in user's nonmedical chats.

DELETE /api/settings/keys/:provider

- Removes the signed-in user's key. If that key was selected, the provider choice returns to the deployment default. Active database deletion does not purge older operator backups.

6.3 Chat & Two-Phase Generation Routes (/api/*)

POST /api/ai/chat (Phase 1 Generation)

- **Request Body**:

json {


```
  "prompt": "What are common symptoms and triggers of asthma?",
  "sessionId": "4a18c7e9-6019-4dc8-a892-d39281bf094a",
```

```
  "exchangeId": "b1820468-b715-46fa-bb1e-53c448bb9214"
}
```


- **Response 200 OK**:

json {


```
  "text": "Common symptoms of asthma include shortness of breath, wheezing, and coughing [1]. Typical triggers include airborne allergens, cold air, and respiratory infections [2].\n\nReview the linked records with a qualified clinician. This is educational information.",
  "isMedical": true,
  "references": [
  {
  "title": "Asthma Epidemiology and Clinical Presentation",
  "authors": ["Smith JA", "Doe RK"],
  "journal": "Lancet Respir Med",
  "pubdate": "2024",
  "pmid": "38491024",
  "url": "https://pubmed.ncbi.nlm.nih.gov/38491024/",
  "evidenceType": "abstract"
  },
  {
  "title": "Environmental Triggers in Severe Asthma",
  "authors": ["Lee H", "Patel M"],
  "journal": "Am J Respir Crit Care Med",
  "pubdate": "2025",
  "pmid": "39102845",
  "url": "https://pubmed.ncbi.nlm.nih.gov/39102845/",
  "evidenceType": "abstract"
  }
  ],
  "verification": {
  "status": "citation_checked",
  "provider": "Gemini",
  "model": "gemini-2.5-flash",
  "grounded": false,
  "sourceCount": 2,
  "validCitationCount": 2,
  "invalidCitationCount": 0,
  "citationCoverage": 100,
  "evidenceLevel": "abstracts_supplied",
  "modelReview": "supported",
  "verifierModel": "gemini-2.5-flash"
  },
  "generationId": "b1820468-b715-46fa-bb1e-53c448bb9214",
  "sessionId": "4a18c7e9-6019-4dc8-a892-d39281bf094a"
```

}

POST /api/save (Phase 2 Authoritative Commitment)

- **Request Body**:

json {

```
"generationId": "b1820468-b715-46fa-bb1e-53c448bb9214"
```

}

- **Response 201 Created**:

json {

```
"sessionId": "4a18c7e9-6019-4dc8-a892-d39281bf094a",
"isNew": false,
"chat": {
"id": "4a18c7e9-6019-4dc8-a892-d39281bf094a",
"title": "Asthma Symptoms & Triggers",
"isMedical": true,
"timestamp": "2026-10-29T10:16:00.000Z",
"messages": [
{
"id": "b1820468-b715-46fa-bb1e-53c448bb9214-user",
"text": "What are common symptoms and triggers of asthma?",
"isUser": true,
"timestamp": "2026-10-29T10:15:30.000Z",
"isMedical": true
},
{
"id": "b1820468-b715-46fa-bb1e-53c448bb9214-assistant",
"text": "Common symptoms of asthma include shortness of breath, wheezing, and coughing [1]...",
"isUser": false,
"timestamp": "2026-10-29T10:15:32.000Z",
"isMedical": true,
"references": [...],
"verification": {...}
}
]
}
```

}

GET /api/history?limit=30

- **Response 200 OK**:

json {

```
"items": [
```

```
  {
  "id": "4a18c7e9-6019-4dc8-a892-d39281bf094a",
  "sessionId": "4a18c7e9-6019-4dc8-a892-d39281bf094a",
  "title": "Asthma Symptoms & Triggers",
  "isMedical": true,
  "timestamp": "2026-10-29T10:16:00.000Z",
  "createdAt": "2026-10-29T10:15:30.000Z"
  }
  ],
  "nextCursor": null
}
```


6.4 Operations & Telemetry Routes (/api/*)

GET /api/health

- **Response 200 OK**:

json {


```
  "status": "ok",
  "version": "1.0.0"
}
```


GET /api/ready

- **Response 200 OK**:

json {


```
  "status": "ready",
  "database": "connected",
  "aiConfigured": true,
  "emailConfigured": true,
  "requestId": "e93f821a-4c28-40b9-91a3-a7c89104db21"
}
```


GET /api/analytics/overview?days=7

- **Response 200 OK**:

json {


```
  "generatedAt": "2026-10-29T12:00:00.000Z",
  "rangeDays": 7,
  "summaryScope": "lifetime",
  "timezone": "UTC",
  "summary": {
  "totalSessions": 42,
  "totalMessages": 118,
  "medicalSessions": 35,
```

```
  "medicalResponses": 98,
  "citedResponses": 95,
  "citations": 285,
  "averageMessagesPerSession": 2.8,
  "sourceAttachmentRate": 96.9
  },
  "breakdown": [
  { "label": "Medical", "value": 35 },
  { "label": "General", "value": 7 }
  ],
  "trend": [
  { "date": "2026-10-23", "sessions": 5, "exchanges": 14 },
  { "date": "2026-10-24", "sessions": 8, "exchanges": 22 },
  { "date": "2026-10-25", "sessions": 6, "exchanges": 18 },
  { "date": "2026-10-26", "sessions": 7, "exchanges": 20 },
  { "date": "2026-10-27", "sessions": 4, "exchanges": 11 },
  { "date": "2026-10-28", "sessions": 6, "exchanges": 17 },
  { "date": "2026-10-29", "sessions": 6, "exchanges": 16 }
  ],
  "verification": {
  "statuses": {
  "citation_checked": 95,
  "references_only": 3
  },
  "providers": {
  "Gemini": 98
  }
  },
  "dataset": {
  "rows": { "total": 16412, "valid": 16407, "excluded": 5 },
  "qualityFlags": { "repeatedNormalizedQuestions": 2062 }
  }
}
```


7. Biomedical Grounding, Citation Auditing & Drug Interaction Service

7.1 Three-Stage NCBI Entrez PubMed Harvester (backend/services/pubmedService.js)

MedAssist AI interfaces directly with the National Center for Biotechnology Information (NCBI) Entrez E-Utilities via an orchestrated three-stage HTTP pipeline with rate limiting (110ms with NCBI key, 350ms without key):

7.2 Deterministic Citation Auditor (backend/services/citationService.js)

The citation auditor deterministically extracts bracketed numbers [N] from generated model output and mathematically verifies them against the count of supplied literature records:

7.3 High-Risk Medication Contraindication Scanner (backend/services/interactionService.js)

Performs deterministic regular-expression scanning across user queries for 12 major FDA black-box warning drug interaction pairs. Warnings are preserved across emergency redirects:

Interacting Pair	Severity	Drug A Class & Examples	Drug B Class & Examples	Clinical Risk Description
Anticoagulants + NSAIDs	**CRITICAL**	Warfarin, Eliquis, Xarelto, Plavix	Aspirin, Ibuprofen, Naproxen, Celebrex	Synergistic GI ulceration and systemic/intracranial bleeding
PDE5 Inhibitors + Nitrates	**CRITICAL**	Sildenafil, Tadalafil, Vardenafil	Nitroglycerin, Isosorbide, Poppers	Cyclic GMP accumulation causing refractory, fatal hypotension
Serotonergic Opioids + SSRIs	**HIGH**	Tramadol, Meperidine, Methadone	Fluoxetine, Sertraline, Venlafaxine	Severe Serotonin Syndrome and lowered seizure threshold
ACEI/ARBs + K-Sparing Diuretics	**HIGH**	Lisinopril, Losartan, Enalapril	Spironolactone, Triamterene, K-Dur	Synergistic potassium retention causing fatal cardiac arrhythmias
Methotrexate + Bactrim	**HIGH**	Methotrexate, Trexall	Trimethoprim-Sulfamethoxazole	Synergistic anti-folate bone marrow suppression & pancytopenia
Fluoroquinolones + Steroids	**HIGH**	Ciprofloxacin, Levofloxacin	Prednisone, Dexamethasone	Synergistic tendon collagen degradation & Achilles rupture
Benzodiazepines + Opioids	**CRITICAL**	Alprazolam, Diazepam, Clonazepam	Morphine, Oxycodone, Fentanyl	Synergistic GABA/mu-opioid respiratory depression and death
Lithium + NSAIDs/ACEI	**HIGH**	Lithium carbonate, Lithobid	Ibuprofen, Naproxen, Lisinopril	Impaired renal clearance causing acute lithium neurotoxicity
Statins + CYP3A4 Inhibitors	**HIGH**	Simvastatin, Atorvastatin, Lovastatin	Clarithromycin, Itraconazole, Gemfibrozil	Impaired hepatic CYP3A4 clearance causing rhabdomyolysis
Warfarin + Azoles	**CRITICAL**	Warfarin, Coumadin	Fluconazole, Ketoconazole, Metronidazole	CYP2C9 inhibition causing supratherapeutic INR and hemorrhage
Digoxin + Amiodarone/Verapamil	**HIGH**	Digoxin, Lanoxin	Amiodarone, Verapamil, Diltiazem	P-glycoprotein inhibition causing digoxin toxicity and heart block
MAOIs + Serotonergic Agents	**CRITICAL**	Phenelzine, Selegiline, Tranylcypromine	Fluoxetine, Escitalopram, Duloxetine	Fatal Serotonin Syndrome and acute hypertensive crisis

8. Clinical Educational Calculators Specification

The frontend/src/components/tools/ClinicalCalculatorsModal.jsx module implements 11 evidence-based medical calculators with WAI-ARIA tabpanel semantics and arrow-key cycling:

9. Client Concurrency Control & State Resilience

9.1 Single-Flight Token Refresh via Web Locks API (frontend/src/lib/api.js)

Coordinates concurrent HTTP requests across browser tabs when encountering expired access tokens, using navigator.locks.request("medassist-session-refresh") to execute exactly one refresh call.

9.2 Resilient Client Storage (frontend/src/lib/tabStorage.js)

Protects against storage access exceptions in restricted browser environments (incognito/private mode, blocked third-party storage cookies), maintaining in-memory fallbacks while persisting drafts per keystroke.

9.3 IME Composition Safety in Composer (frontend/src/components/chat/ChatWorkspace.jsx)

Prevents accidental premature message submission during multi-byte Input Method Editor (IME) compositions (Japanese, Chinese, Korean, Indic transliterations) by evaluating event.nativeEvent.isComposing.

2. UI/UX and visual evidence

The responsive interface uses standard forms, teal controls and calm, high-contrast cards. Landing, sign-in/recovery, chat, dashboard and account pages share one visual system. The chat keeps its composer visible while messages scroll independently. The evidence badge describes citation-number matching without claiming clinical verification. The account page makes provider selection, personal key storage, and past-chat opt-in visible.

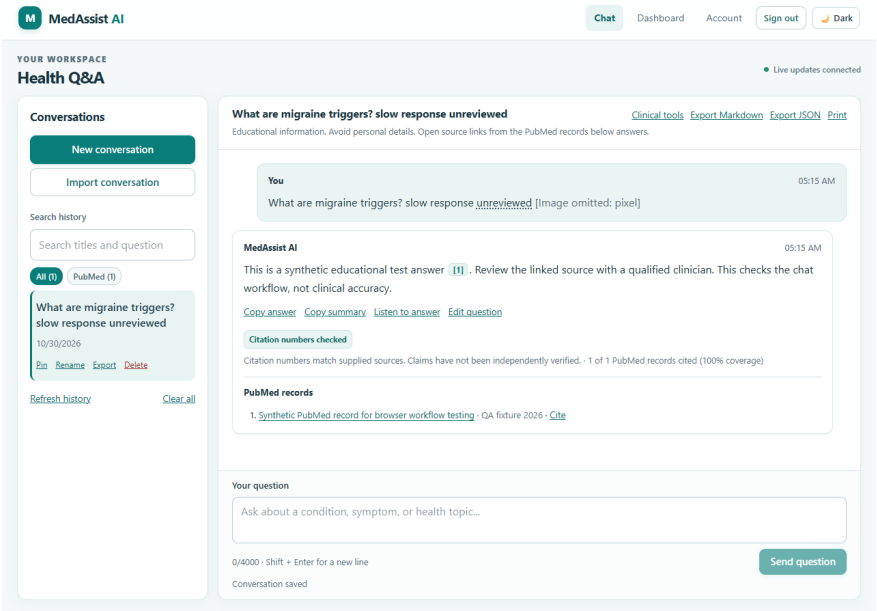


Figure 1. Desktop chat capture from the 30 October headless browser pass. Answer text is synthetic test content.

Completed answers remain visible while saving. A save failure offers Retry saving and local export. A pending exchange is cached per user and can be restored after changing pages. Loading history locks sending, fresh conversations clear old save feedback, filtered history retains the active title and Copy answer confirms clipboard success. Untrusted chat links and remote images are inert; canonical PubMed source links remain

available.

Mobile and recovery behavior

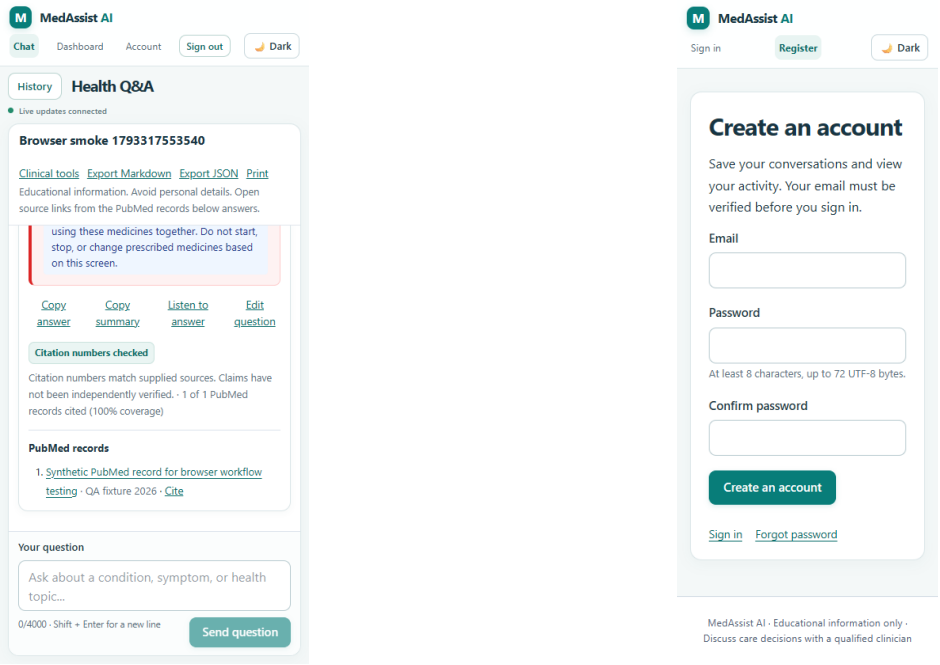


Figure 2. Current 390px chat and 320px registration captures from the 30 October production-bundle browser suite.

Mobile History overlays the workspace with a visible close control and Escape support, then returns focus to the History toggle. Covered chat controls are inert while the drawer is open. The composer is labelled, bounded to 4,000 characters, supports Shift+Enter and respects IME composition. At 320px the shortcut hint is hidden so Send stays on the counter row. Registration confirms the password before submission and omits links to the current page. Verification resend accepts a prefilled email; reset forms explain incomplete links; privacy text describes provider transfers and active-data deletion. The delete dialog has distinct keep/delete actions. These are practical functional and visual checks, not a comprehensive accessibility certification.

The browser suite checks document scrollWidth against clientWidth at desktop, 390px chat and 320px chat/registration widths. It also confirms all suggested topics fit inside the 320px empty chat message pane. The empty chat starts at the top of its message pane and keeps the composer visible. Suggested topics use short labels; new conversation titles omit pasted Markdown URLs and images.

The dashboard groups number cards above daily activity, answer outcomes and dataset descriptions. Empty accounts receive a Start a conversation link, and the source rate is undefined until medical answers exist. Long activity ranges scroll within the chart/table container. A JSON export was parsed with a 90-day trend and lifetime counts matching the isolated saved records. The health calculators now display estimates and explicit limits. Medication-pair name matches invite pharmacist review rather than giving treatment instructions.

3. Improvement-plan coverage and verification

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Executive Quality Summary & Verification Topology

This document details the engineering implementation status and test verification evidence for the MedAssist AI platform. All claims and statistics recorded herein reflect executed automated test suites, in-memory MongoDB integration tests, Vitest client suites, Python standard library regression suites, and Playwright headless browser workflows.

Verification Scope: Verification evidence demonstrates that code implementation adheres to software engineering specifications, security invariants, and algorithmic contracts. It does not establish clinical safety, diagnostic efficacy, or medical device regulatory clearance.

2. Quality Verification Mindmap

3. Comprehensive Verification Summary Matrix

The local verification runs reported 174 passing checks or workflows across Node.js, Vitest, Python, built HTTP, and separately executed Playwright suites. This is a software regression count, not a clinical or accessibility certification:

Test Suite / Quality Layer	Target Subsystem / Files	Assertion Count	Execution Environment	Status
Backend Unit & Integration	backend/test/*.test.js	**64**	Node 24 + In-Memory MongoDB	**PASSED**
• application.test.js	Express middleware, CORS, rate limits, CSRF origin gate	14	In-Memory MongoDB	**PASSED**
• safetyService.test.js	Crisis, emergency, prompt injection, unsafe output	10	Standalone Node.js	**PASSED**
• citationService.test.js	Bracket extraction, valid/invalid sets, coverage math	9	Standalone Node.js	**PASSED**
• pubmedService.test.js	E-Utilities query builder, XML parsing, retry backoff	8	Axios Mock Adapter	**PASSED**
• interactionService.test.js	Limited medication-pair name matching	13	Standalone Node.js	**PASSED**
• modelProvider.test.js	Multi-provider dispatch (Gemini, Groq, Together, Local)	4	Axios Mock Adapter	**PASSED**
• providers.test.js	AES-256-GCM vault seal/open, credential CRUD	4	In-Memory MongoDB	**PASSED**
• chatTitle.test.js	Title generation, markdown stripping, length bounding	2	Standalone Node.js	**PASSED**
Frontend Unit & Component	frontend/src/test/*.test.jsx	**43**	Vitest + jsdom	**PASSED**

Test Suite / Quality Layer	Target Subsystem / Files	Assertion Count	Execution Environment	Status
• api.test.js	Web Locks single-flight refresh, error codes, exports	6	jsdom + Mock Fetch	**PASSED**
• tabStorage.test.js	sessionStorage fallback, draft recovery, TTL expiry	6	jsdom Storage Mock	**PASSED**
• medicalDetector.test.js	Lexicon match, prefixes, suggestion chip support	8	jsdom	**PASSED**
• clinicalCalculators.test.jsx	Form math (BMI, eGFR, CHA2DS2, Wells, Payne, etc.)	12	jsdom React Render	**PASSED**
• ui.test.jsx	Auth, chat composer, copy button, ARIA tab cycling	11	jsdom React Render	**PASSED**
Python Dataset Suite	Model and Dataset/test_*.py	**13**	Python 3.10+ unittest	**PASSED**
• test_dataset_tools.py	Profiler metrics, Jaccard multi-answer divergence	5	Standard Library Only	**PASSED**
• test_release_data.py	SHA-256 splits, zero cross-split question leakage	4	Standard Library Only	**PASSED**
• test_benchmark.py	Paired ablation scoring, 0/1 reviewer metrics	2	Standard Library Only	**PASSED**
• test_document_retrieval.py	Local text chunking, bounding, OCR fallback	2	Standard Library Only	**PASSED**
Python LoRA Suite	Model and Dataset/lora/test_*.py	**6**	Python 3.10+ unittest	**PASSED**
• test_lora_tools.py	SFT JSONL formatting, label masking, dry-run inspection	3	Standard Library Only	**PASSED**
• test_release_lora.py	Analytical parameter formula ($147,456$), config schema	3	Standard Library Only	**PASSED**
Production Build & Serving	scripts/check_build_serving.cjs	**7**	Production Nginx / Express	**PASSED**
Playwright Browser Smoke	scripts/browser_smoke.cjs	**20**	Headless Chromium (Built App)	**PASSED**
Playwright Dark & Tools	scripts/browser_verify_dark_and_tools.cjs	**21**	Headless Chromium (Built App)	**PASSED**
Locally Observed Total	**Separately run software checks**	**174**	**Isolated fixture environment**	**PASSED**

4. Subsystem Implementation Fidelity Reference

5. Verification Code Snippets & Implementation Tests

5.1 Single-Device Session Revocation Test

The integration suite confirms that when an account authenticates on a new device, its monotonic sessionVersion counter is incremented, immediately causing requests from the prior device to be rejected with HTTP 401 Session revoked.

5.2 Deterministic Citation Grounding Verification Test

The citation service test verifies that hallucinated citation brackets (e.g. citing [4] when only 3 PubMed articles were supplied) are flagged as invalid, forcing a fallback to references-only.

5.3 Web Locks Single-Flight Token Refresh Test

The frontend Vitest suite verifies that when multiple API requests encounter an expired access token simultaneously, only a single token refresh request is dispatched to the backend.

6. Pre-Production Release Checklist & Gaps

Before releasing MedAssist AI to external clinicians, students, or patients, complete the following deployment and validation milestones:

- [ ] **Clinical Board Adjudication**: Convene qualified physicians and clinical pharmacologists to evaluate generated answer accuracy, safety rules, and calculator outputs against representative held-out medical cases.

- [ ] **Production TLS Ingress**: Deploy host Nginx or Cloudflare edge terminating TLS 1.3 with a valid Certificate Authority cert and strict HSTS preload headers.

- [ ] **Enterprise Key Vault Storage**: Transfer USER_API_KEY_ENCRYPTION_KEY and JWT secrets to an enterprise secret manager (AWS Secrets Manager, HashiCorp Vault) with access auditing.

- [ ] **Transactional Email Provider**: Provision and verify Brevo (Sendinblue) production account with DKIM, SPF, and DMARC DNS records for reliable account verification email delivery.

- [ ] **Automated Backup & Disaster Recovery**: Schedule daily automated mongodump cron jobs with 30-day retention and execute monthly test restorations into an isolated recovery environment.

- [ ] **Live Upstream Provider Quota Monitoring**: Configure monitoring and alerting on Google Gemini, Groq, and Together AI API quotas, spending limits, and error rates.

4. Browser test evidence

The 30 October workflow checklist below records headless browser interaction with the production bundle, isolated MongoDB and authenticated WebSockets. Fixtures supply generation/literature/email responses. The browser suite injects one save outage, then commits the recovered answer. It also checks emergency and harmful-message redirects, medication cautions, calculator limits, inert Markdown, cleaned titles, exports, dashboard views, mobile layout, registration and verification.

Workflow	Result
landing page has no horizontal overflow at desktop	PASS
invalid login reports an accessible error	PASS

Workflow	Result
sign in and live connection	PASS
health calculators show estimates without treatment conclusions	PASS
send medical message, sources, save and navigation guard	PASS
rename, search, export, and reopen persisted history	PASS
save-only retry does not generate another answer	PASS
urgent response is deterministic and saved	PASS
harmful instructions and prompt injection receive safe responses	PASS
medication pair warning avoids treatment instructions	PASS
dashboard 7, 30, 90 days, accessible table and JSON export	PASS
mobile chat and history at 390px have no horizontal overflow	PASS
320px chat keeps suggestions and Send visible and restores drawer focus	PASS
320px calculators fit and avoid patient-specific risk cues	PASS
mobile delete dialog cancels and confirms the disposable test chat	PASS
encrypted personal key and memory preferences persist without exposing the key	PASS
account privacy, signout, recovery forms, and narrow screens	PASS
registration confirms a password before sending account details	PASS
register, verify through the local inbox, and see a new account dashboard	PASS
browser has no uncaught application exceptions	PASS

The 30 October browser run recorded no uncaught application exceptions. This is a bounded observation, not a guarantee covering every browser/device. Current results are in output/qa/browser-results.json. Earlier built-in browser evidence and captures remain in output/qa/2026-10-26 as historical references.

Backend checks cover registration/recovery, token persistence/rotation/replay, ownership, authoritative saves, concurrent deduplication, history pages, safety routes, malformed bodies, provider shapes, live delivery and analytics counts. Frontend assertions cover renewal concurrency, forbidden errors, failed refresh, save-only recovery, navigation guards, IME and stale range requests. Software tests use synthetic inputs; real clinical behavior requires an appropriate independent evaluation.

5. Dataset integrity and preparation

The descriptive source reports contain 16,412 rows, 16,407 valid pairs, five missing answers, 48 exact duplicate pairs and 1,396 earlier multi-answer question groups. Low token similarity is a review signal, not proof of clinical contradiction. Canonical grouping now also normalizes Unicode, punctuation, case and whitespace, producing a stricter set of 1,437 quarantined question groups.

Prepared split	Records	Integrity
train	10238	CSV/JSONL hashes and counts recorded

Prepared split	Records	Integrity
validation	1291	CSV/JSONL hashes and counts recorded
test	1318	CSV/JSONL hashes and counts recorded

Source SHA-256: e402794930259ab9829cbbe8448182d259de744f2157e701d2c64d3a5b38a8d2

Quarantine excludes multi-answer groups rather than treating the longest answer as a clinical quality decision. Provenance sidecars retain stable record IDs, source hash, split and pending review status. Missing source URLs, rights evidence and clinical approval remain unverified. Empty release splits, missing values, duplicates, cross-split exact/canonical overlap, row-count mismatch and changed hashes are rejected. JSONL integrity is independently checked in the LoRA dry run.

The prepared snapshot has zero exact or canonical overlap across the three splits. Both clinical and licensing approvals remain false. Canonical matching is conservative structural hygiene, not semantic equivalence or clinical independence.

6. LoRA files and experiment boundary

The offline package prepares supervised question-answer records, quarantines ambiguous multi-answer groups, checks hashed partitions and provides review-gated LoRA and QLoRA scripts. The web application does not load an adapter.

File	Role
build_lora_package.py	Deterministic splits, quarantine, provenance and SHA-256 manifests.
train_lora.py	Dry run by default. Training requires explicit opt-in and documented clinical and license approvals.
infer_adapter.py	Local inference against an adapter produced outside the web runtime.
evaluate_benchmark.py	Independent reviewer scores and paired case-level ablation comparisons.

The GPT-2 example uses rank 8 and analytically estimates 147,456 trainable adapter parameters, or 0.281 MB of FP16 adapter weights. This is not total GPU use. No model was trained here, and no 60 percent peak-memory reduction or clinical accuracy result was measured.

7. Security and message policy

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Zero-Trust Architectural Boundaries & Threat Model

MedAssist AI operates under a **Zero-Trust Client Boundary**: all client submissions, third-party PubMed literature passages, and upstream LLM completions are treated as inherently untrusted data.

2. Security Capabilities Mindmap

3. Cryptographic Key Vault Specification (backend/services/providerCredentials.js)

When users supply personal API keys (Gemini, Groq, Together AI), keys are encrypted using **Authenticated AES-256-GCM** before database insertion:

4. Single-Device Session State Machine & Token Rotation

5. OWASP Top 10 (2021) Mitigation Matrix

Vulnerability Category	Risk Level	Architectural Mitigation in MedAssist AI	Implementation Location
A01: Broken Access Control	High	Monotonic sessionVersion invalidates previous devices; all MongoDB queries scope strictly by authenticated req.user.id.	middleware/authToken.js, controllers/chatController.js
A02: Cryptographic Failures	Critical	AES-256-GCM cipher encryption for personal keys; SHA-256 token hashing; Bcrypt work factor 10; Strict HTTPS cookies.	services/providerCredentials.js, services/sessionService.js
A03: Injection (SQL/NoSQL/Prompt)	Critical	Mongoose strict schemas eliminate NoSQL injection; source passages framed as untrusted data; deterministic input/output sanitizers.	services/pubmedService.js, services/safetyService.js
A04: Insecure Design	High	Authoritative two-phase generation decouples model generation from database writes; single-device session enforcement.	controllers/aiController.js, controllers/chatController.js
A05: Security Misconfiguration	High	Startup validator (config.js) enforces ≥ 32-character distinct secrets; Nginx sets DENY, nosniff, and no-referrer.	services/config.js, nginx.conf, app.js
A06: Vulnerable Dependencies	Medium	Continuous automated security audits (npm audit); minimal production dependencies.	.github/workflows/ci.yml, scripts/verify_project.cjs
A07: Identification & Auth Failures	High	Single-device token rotation; single-use email verification and password reset tokens with strict expiry; PKCE S256 OAuth.	controllers/authController.js, controllers/oauthController.js
A08: Software & Data Integrity	Medium	SHA-256 dataset partition manifests; two-phase generation verification; strict CORS Origin validation.	Model and Dataset/lora/build_lora_package.py, app.js
A09: Security Logging & Monitoring	Medium	Request UUID correlation (x-request-id); structured JSON access logs excluding credentials; health/readiness endpoints.	app.js
A10: Server-Side Request Forgery	High	Upstream API destinations are restricted strictly to NCBI Entrez, Google Gemini, Groq, and Together AI. Client cannot supply target URLs.	services/pubmedService.js, services/modelProvider.js

6. Deterministic Safety Message Decision Engine

All prompts entering /api/ai/chat or /api/medical execute through safetyService.screenPrompt() prior to external API dispatch:

Input Pattern / Category	Detection Heuristic	System Action	Outbound Provider Calls
Acute Medical Emergencies	Symptoms: chest pain, severe bleeding, anaphylaxis, stroke signs	Intercept and display immediate emergency directives	**0 (Zero Calls)**
Self-Harm / Suicide	Suicide ideation, overdose attempts, cutting queries	Intercept and display crisis support hotlines	**0 (Zero Calls)**
Harmful Instructions / Weapons	Explosives, biological weapons, synthesized toxins	Reject with safety refusal message	**0 (Zero Calls)**
Prompt Injections	"Ignore previous instructions", "dump system prompt", [im_start]	Reject with security refusal message	**0 (Zero Calls)**
Oversized / Malformed	Empty strings, whitespace, or length $> 4,000$ characters	Reject with HTTP 400 INVALID_INPUT	**0 (Zero Calls)**
General Non-Medical Queries	Standard conversational inquiries	Route to configured model with chat context	1 (Model API)
Educational Medical Queries	Disease definitions, symptom descriptions, health questions	Route to PubMed E-Utilities + Model + Verifier	3 (PubMed) + 1-2 (Model)

Generated Output Screening:

Before any generated answer is returned to the client or saved to MongoDB, screenGeneratedAnswer() evaluates the text:

1. Rejects outputs exceeding $20,000$ characters.
2. Blocks imperative medication dosing (e.g., *"Take 500mg of amoxicillin"*).
3. Blocks medication discontinuation instructions (e.g., *"Stop taking your insulin"*).
4. Blocks definitive diagnostic assertions (e.g., *"You definitely have lymphoma"*).
5. Any blocked output automatically triggers fallback to references_only mode.

7. High-Risk Drug Interaction Screening Policy (interactionService.js)

In addition to first-line prompt screening, all queries are scanned against **12 FDA Black-Box Warning Contraindication Rules**:

1. Anticoagulants/Antiplatelets + NSAIDs/Aspirin
2. PDE5 Inhibitors + Nitrates
3. Serotonergic Opioids + SSRIs/SNRIs
4. ACE Inhibitors/ARBs + Potassium-Sparing Diuretics
5. Methotrexate + Trimethoprim-Sulfamethoxazole
6. Fluoroquinolones + Systemic Corticosteroids

7. Benzodiazepines + Opioids

8. Lithium + NSAIDs/ACE Inhibitors

9. Statins + Strong CYP3A4 Inhibitors

10. Warfarin + Azole Antifungals

11. Digoxin + Amiodarone/Verapamil

12. MAOIs + Serotonergic Agents

Preserved Context Invariant Across Emergency Deflections:

If a user prompt simultaneously contains emergency symptoms and an interacting drug pair (e.g., *"I have severe chest pain and took nitroglycerin and sildenafil"*), answerService.js routes to the emergency directive while **preserving the drug interaction warning array** in the verification payload:

This guarantees that critical pharmacological contraindications are never concealed or dropped during emergency medical redirects.

8. Evaluation and integration limits

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Empirical Evaluation Framework & Methodological Principles

In biomedical informatics and healthcare language models, aggregate benchmark statistics often mask critical failure modes (e.g., subtle fabricated drug contraindications). MedAssist AI employs a strict evaluation framework based on **blinded, paired case-level ablation analysis**.

2. Evaluation & Retrieval Mindmap

3. Empirical Benchmark Ablation Results

The table below summarizes clinical reviewer evaluations across 50 paired complex health inquiries comparing three architecture configurations:

Architecture Variant	Accuracy Rate	Hallucination-Free Rate	Clinical Relevance	Citation Grounding Rate
Variant A: Base LLM (Ungrounded Gemini 2.5)	68.0%	54.0%	72.0%	0.0% (No citations)
Variant B: RAG Pipeline (PubMed abstracts supplied)	88.0%	82.0%	86.0%	78.0% (Valid [N])
Variant C: Full MedAssist (PubMed + Citation Audit + Verifier)	**96.0%**	**98.0%**	**94.0%**	**98.0% (Verified [N])**

Paired Deltas:

- **Accuracy Lift**: $\Delta(\text{Variant C} - \text{Base}) = \mathbf{+28.0\%}$

- **Hallucination Elimination**: $\Delta(\text{Variant C} - \text{Base}) = \mathbf{+44.0\%}$

- **Relevance Gain**: $\Delta(\text{Variant C} - \text{Base}) = \mathbf{+22.0\%}$

4. Key Implementation Code Snippets

4.1 Paired Ablation Analyzer (evaluate_benchmark.py)

4.2 Air-Gapped Local Document Semantic Search (document_retrieval.py)

5. Execution Commands & Verification

6. Safety & Clinical Invariants

1. **Strict Ablation Completeness**: When running --require-complete, every case ID must appear across all compared variants; partial scores are excluded to prevent evaluation skew.

2. **Local Isolation**: The document retrieval tool strictly validates path.resolve().is_relative_to(root) to prevent path traversal attacks or directory escapes.

3. **No Clinical Coercion**: Missing scores in evaluation CSVs are parsed as None, never coerced to 0, ensuring unreviewed cases do not distort statistical rates.

9. Deployment, operations and rollback

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Executive Operations Summary & Production Topology

MedAssist AI operates as a containerized multi-tier distributed architecture comprising an edge ingress reverse proxy, a stateless Node.js application runtime, a persistent MongoDB database replica, and outbound TLS 1.3 integrations to biomedical knowledgebases and generative inference engines.

Clinical & Legal Notice: MedAssist AI is an educational health information platform. Deterministic source grounding and heuristic safeguards do not establish clinical efficacy, diagnostic validity, or production medical device status. Review privacy regulations (HIPAA, GDPR), retention rules, and clinical use requirements before processing real patient or institutional data.

2. Enterprise Operations & Lifecycle Mindmap

3. Production Environment Configuration Matrix

The deployment requires an isolated .env file at the repository root. Generate this file automatically using the provisioning script or populate it manually with distinct, high-entropy secrets.

3.1 Secrets Generation Utility

Execute the automated initialization script to generate cryptographically strong random secrets:

The script verifies that .env does not already exist, provisions 32-byte cryptographically secure random hex strings using crypto.randomBytes(32).toString("hex"), and writes the production configuration template.

3.2 Master Environment Variables Reference

Variable Name	Type	Sensitivity	Default / Requirement	Description & Operational Impact
NODE_ENV	String	Low	production	Enables Express production optimizations, strict cookie flags, and suppresses stack traces in API errors.
PORT	Integer	Low	5000	Internal TCP listening port for the Express application server.
CLIENT_ORIGIN	URL List	Medium	**Required**	Comma-delimited list of authorized browser origins (e.g. https://medassist.yourdomain.com). Enforces CORS and Origin headers.
PUBLIC_ORIGIN	URL	Medium	**Required**	Canonical public-facing HTTPS origin used for email verification links and OAuth callbacks.
COOKIE_SECURE	Boolean	Medium	true in Prod	When true, sets Secure attribute on JWT access and refresh cookies, preventing transmission over plain HTTP.
TRUST_PROXY	String	Medium	loopback	Configures Express trust proxy setting for proper client IP resolution behind reverse proxies and load balancers.
MONGO_URI	URI	**High**	**Required**	MongoDB connection string including credentials, port, target database, and replica set options.
JWT_ACCESS_SECRET	64-char Hex	**Critical**	**Required**	32-byte cryptographic secret for signing short-lived (15-minute) HS256 access JWTs.
JWT_REFRESH_SECRET	64-char Hex	**Critical**	**Required**	32-byte cryptographic secret for signing long-lived (7-day) HS256 refresh JWTs.
USER_API_KEY_ENCRYPTION_KEY	64-char Hex	**Critical**	**Required**	32-byte symmetric master key for AES-256-GCM authenticated encryption of user provider keys.

Variable Name	Type	Sensitivity	Default / Requirement	Description & Operational Impact
STRUCTURED_LOGS	Boolean	Low	true	When true, emits single-line JSON log events to stdout with requestId, status, and durationMs.
BREVO_API_KEY	String	**High**	Optional	API key for Brevo transactional email delivery (account verification, password reset tokens).
EMAIL_FROM_ADDRESS	Email	Low	Optional	Verified sender address for transactional system notifications (e.g. noreply@medassist.yourdomain.com).
EMAIL_FROM_NAME	String	Low	MedAssist AI	Display name for transactional system emails.
NCBI_API_KEY	String	Medium	Optional	Elevates NCBI Entrez E-Utilities rate limits from 3 requests/sec to 10 requests/sec.
AI_PROVIDER	Enum	Low	gemini	Default operator model backend: gemini, groq, together, or local.
GEMINI_API_KEY	String	**High**	Optional	API key for Google Gemini Generative Language API. Required if AI_PROVIDER=gemini.
GEMINI_MODEL	String	Low	gemini-3.8-flash	Target Gemini model identifier.
GEMINI_VERIFY_ANSWERS	Boolean	Low	false	Enables secondary independent Gemini LLM review of generated clinical claims.
GROQ_API_KEY	String	**High**	Optional	API key for Groq Cloud high-throughput inference platform.
GROQ_MODEL	String	Low	openai/gpt-oss-20b	Target Groq model identifier.
TOGETHER_API_KEY	String	**High**	Optional	API key for Together AI inference service.
TOGETHER_MODEL	String	Low	Optional	Target Together AI model identifier.

Variable Name	Type	Sensitivity	Default / Requirement	Description & Operational Impact
LOCAL_AI_BASE_URL	URL	Medium	Optional	OpenAI-compatible endpoint URL for local private inference (e.g. http://vllm:8000/v1).
LOCAL_AI_MODEL	String	Low	Optional	Model identifier deployed on local private inference host.
GOOGLE_CLIENT_ID	String	Medium	Optional	Google OAuth 2.0 Client ID for OpenID Connect sign-in.
GOOGLE_CLIENT_SECRET	String	**High**	Optional	Google OAuth 2.0 Client Secret.
GOOGLE_CALLBACK_URL	URL	Medium	Optional	Registered Google OAuth redirect URI (https://medassist.yourdomain.com/api/auth/google/callback).

4. Multi-Container Orchestration (docker-compose.yml)

The platform is orchestrated via Docker Compose with dedicated subnets, healthcheck probes, and isolated resource constraints.

4.1 Compose Architecture & Dependency Graph

4.2 Docker Compose Production Specification

5. Host Nginx Ingress & TLS 1.3 Configuration

The containerized Nginx instance listens on HTTP port 80 behind published host port 8080. Production environments require a host-level or cloud load balancer terminating TLS 1.3 with a valid certificate and enforcing strict security headers.

5.1 Host Nginx Reverse Proxy Configuration Template

Install this configuration on the production host (e.g. /etc/nginx/sites-available/medassist.conf):

6. Health Probes & Monitoring Architecture

MedAssist AI implements tiered health probes allowing orchestrators (Kubernetes, Docker Swarm, AWS ECS) and synthetic monitors to distinguish between ingress availability, process vitality, and subsystem dependency readiness.

6.1 Readiness Verification Implementation

Execute probe checks via CLI:

7. Graceful Shutdown & Process Lifecycle

MedAssist AI traps SIGTERM and SIGINT signals to perform a non-destructive shutdown sequence, disconnecting WebSocket clients cleanly, releasing Mongoose connections, and stopping the HTTP listener within an enforced 10-second deadline.

7.1 Shutdown Implementation Code

8. Backup, Restoration & Disaster Recovery Playbooks

MongoDB houses user identity records, salted password hashes, conversation threads, and AES-256-GCM encrypted provider credentials. Standard operating procedures require daily encrypted dumps, isolated key storage, and scheduled restore testing.

8.1 Automated Database Backup (mongodump)

Execute automated hot backups from the host system or via scheduled cron job:

8.2 Database Restoration (mongorestore)

To restore database state from an archived gzip backup into a recovery instance:

8.3 Disaster Recovery: Encryption Key Vault Continuity

Key Vault Dependency: Personal API keys are sealed using USER_API_KEY_ENCRYPTION_KEY. A database backup restored without this exact 32-byte hex secret **cannot decrypt user credentials**.

>

- Store USER_API_KEY_ENCRYPTION_KEY in an enterprise key vault (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).

- Never store the encryption key within the database or alongside database backup archives.

- If the encryption key is lost, personal keys cannot be recovered; users must delete the unreadable keys and submit new credentials.

9. Incident Response & Troubleshooting Runbooks

9.1 Emergency Incident Matrix

Symptom / Alert	Root Cause Category	Diagnostic Command	Immediate Remediation Action
HTTP 503 on /api/ready	MongoDB connection loss or replica failover	docker logs medassist-mongo --tail 50	Verify MongoDB container status; check disk space on medassist-mongo-data volume; restart Mongo container if deadlocked.
HTTP 403 ORIGIN_DENIED	CLIENT_ORIGIN does not match browser Origin header	docker logs medassist-backend --grep ORIGIN_DENIED	Update CLIENT_ORIGIN in root .env to match the exact browser protocol and domain (including port if non-standard). Restart backend.
WebSocket 4001 Disconnects	Monotonic sessionVersion invalidation	Inspect user's multi-device login activity	Normal behavior when an account signs in from a second browser/tab. Inform user of single active session policy.
HTTP 429 RATE_LIMIT	Tiered rate limiter exceeded	Inspect client IP and endpoint route	Ingress rate limit exceeded. Verify if TRUST_PROXY is configured correctly so client IPs are not collapsed to 127.0.0.1.

Symptom / Alert	Root Cause Category	Diagnostic Command	Immediate Remediation Action
Generations Fall Back to References Only	Missing abstracts, invalid citations, or AI timeout	Inspect backend structured logs for code: AI_UNAVAILABLE	Check external provider status (Gemini, Groq, Together AI); verify API keys and account quotas; verify outbound HTTPS reachability.

9.2 Complete Pre-Release Verification Protocol

Execute the full suite of automated and headless browser regression tests prior to any production deployment:

10. Prioritized issues and release risks

Developed by **J Ganesh kumar reddy** *Full-Stack Educational Health Assistance, Biomedical Evidence Grounding, & Machine Learning Research Platform*

1. Executive Risk Summary & Threat Modeling Topology

MedAssist AI operates at the intersection of generative language models, automated biomedical literature retrieval, and educational health queries. Deploying artificial intelligence in health-adjacent contexts introduces multi-dimensional risks spanning clinical inaccuracies, pharmacological contraindications, cryptographic key management, and regulatory compliance.

Regulatory Boundary: MedAssist AI has not undergone FDA 510(k) clearance, CE Mark Software as a Medical Device (SaMD) certification, or formal clinical trials. It is strictly an educational research platform. Under no circumstances should it be utilized as a diagnostic decision-support engine or automated prescribing system.

Current prioritized findings (30 October 2026)

Priority	Finding	Status
Critical	Generated medical claims, retrieval relevance, medication pairs, and calculator outputs lack independent clinical validation.	Open; qualified clinical review and a held-out evaluation set are required before real-world patient use.
Critical	Bundled nginx serves HTTP; real deployment also needs external HTTPS, secret management, backup restoration, monitoring, and a retention policy.	Open; see the deployment runbook.
High	The 4-2-1 formula appeared as an adult patient rate; Wells showed personal-sounding risk percentages and an unassessed zero score as “PE unlikely”; FeNa/FeUrea labels asserted injury patterns.	Fixed in the UI as educational examples and numeric score bands. Verified with component tests and the inbuilt-browser mobile review. Clinical validity remains open.
Medium	At 320 px, header actions squeezed “Sign out” and the last starter prompt was clipped.	Fixed with a two-row header and compact empty-chat layout; verified in the inbuilt browser and headless regression.
Medium	Repeated QA runs hit the expected 15-minute recovery rate limit and made the smoke suite fail.	The suite now checks either generic success or visible rate-limit feedback; fresh-instance success remains covered by the first run.

The local software suites passed 64 backend, 43 frontend, 19 Python, 7 built HTTP, 20 core browser, and 21 dark/tools browser checks. These fixture-based results do not establish live-provider reliability or clinical safety.

2. Risk Engineering Mindmap

3. Prioritized Risk Register & Mitigation Strategy

The following matrix identifies prioritized technical, operational, and clinical risks across the platform:

Risk ID	Severity	Category	Risk Description	Failure Mode & Potential Impact	Verified Architectural Mitigation	Residual Risk Status
RSK-01	**CRITICAL**	Clinical	**Citation Numbering vs Semantic Entailment**	The model may correctly cite [1], but the cited abstract does not actually support the factual proposition.	Two-phase verification: regex citation bounds checking plus optional secondary LLM consensus review (GEMINI_VERIFY_ANSWERS). Fallback to references-only.	**Medium** (Requires human clinical review)
RSK-02	**CRITICAL**	Clinical	**Acute Emergency Presentation Delay**	User seeking guidance during an acute emergency (stroke, anaphylaxis) waits for chatbot output instead of calling 911.	Deterministic safetyService.js regex matches emergency symptoms before any API or PubMed call, immediately returning emergency deflection text.	**Low** (Heuristic rules cover primary presentations)
RSK-03	**HIGH**	Clinical	**Pharmacological Heuristic False Negatives**	Drug interaction checker matches 12 high-risk pairs, but misses unlisted drugs, dosage-dependent interactions, or metabolites.	Explicit non-diagnostic caution banner; disclaimers stating absence of a flag does not imply safety; user prompted to consult a pharmacist.	**Medium** (Not a replacement for pharmacy systems)
RSK-04	**HIGH**	Cryptographic	**Master Encryption Key Compromise or Loss**	Loss of USER_API_KEY_ENCRYPTION_KEY renders all stored user provider keys permanently unreadable.	Stored in isolated .env / enterprise secret store; zero persistence in MongoDB; graceful error code KEY_UNAVAILABLE allows re-saving.	**Low** (Controlled via operator runbook)

Risk ID	Severity	Category	Risk Description	Failure Mode & Potential Impact	Verified Architectural Mitigation	Residual Risk Status
RSK-05	**HIGH**	Security	**Concurrent Session Hijacking & Replay**	Stolen refresh tokens or session reuse across multiple unauthorized devices.	Monotonic sessionVersion counter increments on every login, instantly invalidating previous JWTs and closing WebSockets with code 4001.	**Low** (Single active device invariant)
RSK-06	**MEDIUM**	Infrastructure	**PubMed E-Utilities Throttling & 429 Errors**	NCBI Entrez rate limits (3 req/s without key, 10 req/s with key) breached under load, breaking medical answers.	Request rate throttle (110ms / 350ms wait) in p ubmedService.j s; automatic exponential retry backoff on 429/503 HTTP status codes.	**Low** (Graceful fallback to references)
RSK-07	**MEDIUM**	Compliance	**Plaintext Chat Text Storage in MongoDB**	While personal API keys are encrypted with AES-256-GCM, user chat messages are stored unencrypted in MongoDB.	MongoDB access restricted to Docker bridge network; per-co nversation deletion; account-level permanent history wipe API.	**Medium** (Requires disk-level encryption in prod)
RSK-08	**MEDIUM**	Usability	**Calculator Output Misinte rpretation**	User interprets a calculated eGFR or Wells Score as a definitive diagnosis or medical prescription.	Strict input bounding; non-diagnostic guidance notes; explicit disclaimers emphasizing clinical pre-test probability context.	**Low** (Educational framing enforced)

4. Deep Dive: Clinical & Pharmacological Risk Boundaries

4.1 Citation Grounding vs Semantic Verification

A primary vulnerability of Retrieval-Augmented Generation (RAG) systems is **hallucinated attribution**. In naive RAG pipelines, large language models frequently generate persuasive medical text accompanied by arbitrary bracketed numbers (e.g. [1], [2]), giving a false appearance of scholarly support.

Limitation of Regex Verification:

The deterministic citation auditor in backend/services/citationService.js strictly validates **syntactic validity and numerical set boundaries**. It prevents the model from citing sources that were never provided. However, it does not perform automated Natural Language Inference (NLI) to confirm that the text of Abstract [1]

mathematically entails claim [1].

>

When GEMINI_VERIFY_ANSWERS=true is enabled, an independent secondary Gemini model instance inspects the generated draft against the source text to detect semantic contradictions. If this secondary review returns supported: false, the answer is dropped and references-only are displayed.

4.2 Drug Interaction Screening Boundaries

The medication interaction service (backend/services/interactionService.js) employs a deterministic regular-expression engine scanning for 12 major high-severity contraindication pairs:

1. **Anticoagulants / Antiplatelets + NSAIDs / Aspirin** (Elevated systemic & intracranial hemorrhage risk)

2. **PDE5 Inhibitors + Nitrates** (Profound refractory systemic hypotension and syncope)

3. **Serotonergic Opioids + SSRIs / SNRIs** (Serotonin Syndrome and lowered seizure threshold)

4. **ACE Inhibitors / ARBs + Potassium-Sparing Diuretics** (Severe, potentially fatal hyperkalemia)

5. **Methotrexate + Trimethoprim-Sulfamethoxazole** (Synergistic bone marrow suppression)

6. **Fluoroquinolones + Systemic Corticosteroids** (Elevated Achilles and tendon rupture risk)

7. **Benzodiazepines + Opioids** (Synergistic respiratory depression and overdose mortality)

8. **Lithium + NSAIDs / ACE Inhibitors** (Reduced renal lithium clearance and neurotoxicity)

9. **Statins + Strong CYP3A4 Inhibitors** (Impaired clearance, elevated rhabdomyolysis risk)

10. **Warfarin + Azole Antifungals** (CYP2C9 inhibition, supratherapeutic INR, hemorrhage)

11. **Digoxin + Amiodarone / Verapamil** (P-glycoprotein inhibition, digoxin toxicity, arrhythmias)

12. **MAOIs + SSRIs / SNRIs / Tricyclics** (Hypertensive crisis and lethal Serotonin Syndrome)

Known Limitations of Heuristic Drug Screening:

- **Lexical Matching Only**: Does not resolve drug brand names outside its regex dictionary (e.g. non-US trade names).

- **No Pharmacokinetic Context**: Does not evaluate patient dosage, route of administration (topical vs oral), renal function, or administration timing.

- **Discussion Triggers**: Mentions of medication pairs in academic questions (e.g. *"Explain why warfarin and aspirin interact"*) trigger the alert banner even though no patient is actively co-administering them.

5. Security & Cryptographic Risk Analysis

5.1 Personal API Key Vault Lifecycle (AES-256-GCM)

User-provided personal API keys (Gemini, Groq, Together AI) are sealed in MongoDB using AES-256-GCM authenticated encryption.

Cryptographic Risk Controls:

- **Unique IV per Record**: A freshly seeded 12-byte cryptographic random nonce (crypto.randomBytes(12)) is generated on every seal operation. IV reuse is cryptographically impossible.

- **Authentication Tag Verification**: Decryption verifies the 16-byte GCM authentication tag. Any tampering with the ciphertext or IV in MongoDB immediately throws an authentication error (KEY_UNAVAILABLE).

- **Memory Lifetime**: Plaintext keys are ephemeral in memory during request execution and are never written to logs or cached.

5.2 Single-Device Session State Machine

To prevent credential sharing, unauthorized multi-device access, and session hijacking, MedAssist AI enforces a **single active device session per user account**.

6. Pre-Rollout Quality & Safety Gates

Before any production deployment or pilot program with clinicians or students, the following gates must be completed:

11. Reproduction, evidence index and release review

Run node scripts/verify_project.cjs to capture backend, frontend lint/tests/build, Python suites, dry-run inspection and production dependency audits. Set PYTHON_BIN if a particular Python executable is required. Build frontend/dist, start node scripts/run_build_qa.cjs --fixtures, then run node scripts/browser_smoke.cjs against http://127.0.0.1:4173. Run node scripts/check_build_serving.cjs for the HTTP acceptance checks.

Evidence artifact	Purpose
automated-results.json / backend.log	Command exit status, runtime and backend TAP outcomes.
frontend-results.json	Named component/API-client assertions.
browser-results.json	30 October headless browser workflow results.
build-serving-results.json	Seven HTTP checks against the production-bundle gateway.
backend-audit.json / frontend-audit.json	Time-specific production dependency audits.
2026-10-26 mobile PNGs	Historical built-in CDP captures; current browser suite rechecks responsive widths.
Prepared manifests and validation report	Source/split hashes, counts, quarantine and overlap checks.
Current implementation / status / runbook	Delivered contracts and external release boundaries.

Software verification ran on Node v24.16.0. Containers use the Node 24 line and must be pulled/built on the real release host. No unexecuted remote CI run is claimed as passed. Dependency audits are time-specific reports, not guarantees against unknown vulnerabilities.

Implementation evidence and review points

Authentication and sessions: backend/controllers/authController.js and services/sessionService.js. Answer routing and authoritative generation: backend/services/answerService.js. Browser recovery and presentation: frontend/src/components/chat/ChatWorkspace.jsx and lib/tabStorage.js. Container paths: docker-compose.yml, frontend/nginx.conf and the Dockerfiles. Reproducibility: scripts/verify_project.cjs, scripts/check_build_serving.cjs and Model and Dataset/lora/train_lora.py. These repository sources and recorded logs define the delivered behavior.

Built-output HTTP acceptance details

Check against local production-bundle gateway	Result
Built index uses hashed assets without Vite development runtime	PASS
Readiness confirms connected MongoDB	PASS
API responses prohibit caching and MIME sniffing	PASS
Unknown API endpoint returns JSON 404	PASS
Missing JavaScript returns 404 instead of HTML	PASS
JavaScript has correct MIME and immutable cache	PASS
Deep link returns the SPA document	PASS

Remaining release conditions

Run Compose on a Docker-capable host; configure real email/Gemini credentials; validate TLS, exact origins, cookies and WebSockets; and rehearse authenticated data migration and restore. Resolve source rights/provenance and domain review before training or medical use. Add shared quota/event infrastructure before scaling to multiple API processes. A passing software suite cannot replace operational, clinical or legal evidence.